

Lab Assessment 3

Software Process Model & Software Project
Management document of the project

Fog-Based Vehicular Monitoring and Autonomous Fault Response System

Slot: L9 + L10

Course Code and Title - BCSE301P, Software Engineering Lab.

Team Members and their roles -

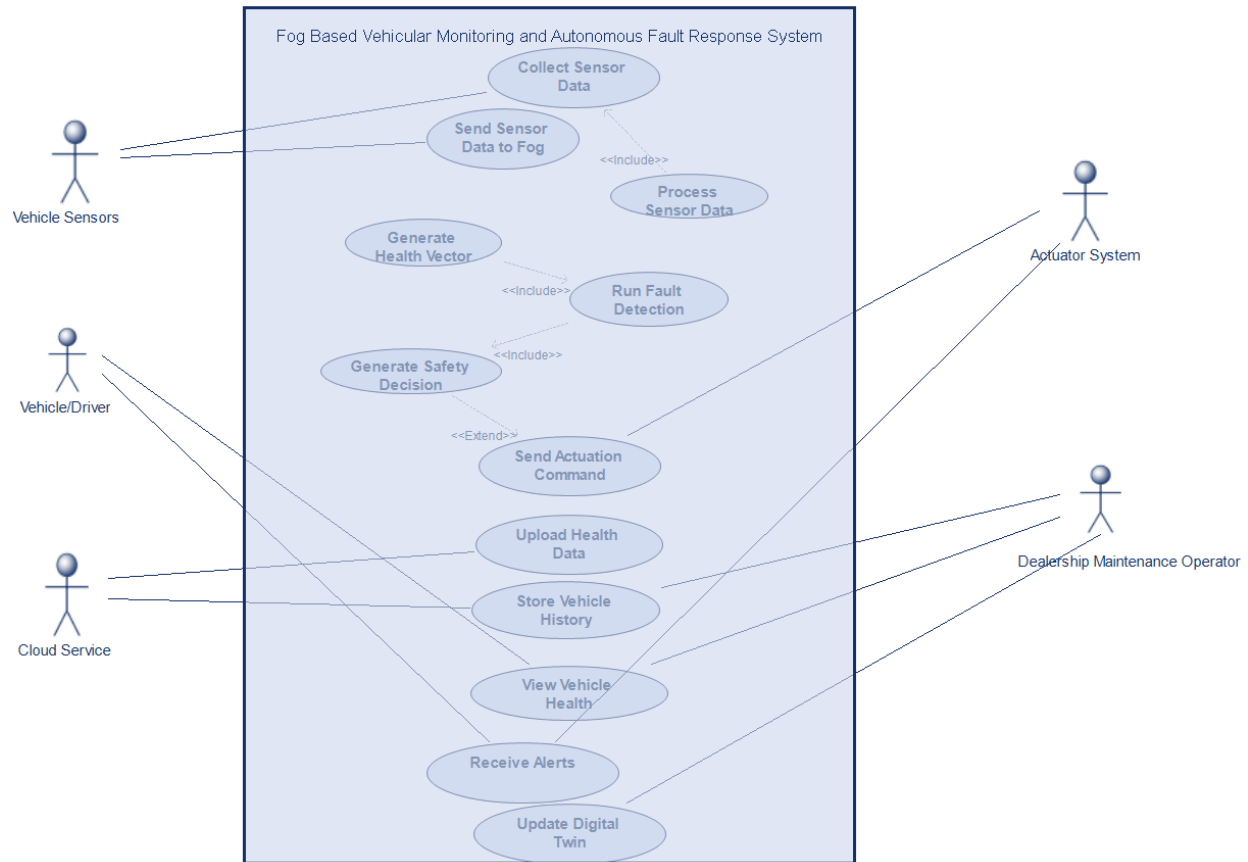
Pushkal Gupta (23BCT0253)—Team Lead, System Architect, Fog Head

Adrivid Mishra (23BCT0264)—Hardware & Embedded Systems

Vaibhav Jain (23BAI0033)—AI & Data Intelligence.

Shagnik Paul (23BCT0266)—UI, Backend, Visualization & Deployment

Use Case Diagram:



[Use Case Diagram Link](#)

Use Case Descriptions:

UC-01: Collect Sensor Data

Field	Description
Use Case ID	UC-01
Use Case Name	Collect Sensor Data
Created By	System Development Team
Last Updated By	System Development Team

Date Created	2026
Date Last Updated	2026
Actor	Vehicle Sensors
Description	The system collects telemetry data such as temperature, tire pressure, battery voltage, and motion readings from vehicle sensors.
Preconditions	Sensors are installed and connected to the edge device.
Postconditions	Raw vehicle sensor data is available for further processing.
Priority	High
Frequency of Use	Continuous / Real-time
Normal Course of Events	1. Sensors detect vehicle parameters. 2. Edge device reads sensor values. 3. Sensor data is captured by the system.
Alternative Courses	Sensor temporarily unavailable → system retries data collection.
Exceptions	Sensor failure detected.
Includes	Send Sensor Data to Fog
Special Requirements	Real-time data acquisition with low latency.
Assumptions	Sensors are properly calibrated.
Notes and Issues	Sensor noise may affect accuracy.

UC-02: Send Sensor Data to Fog

Field	Description
-------	-------------

Use Case ID	UC-02
Use Case Name	Send Sensor Data to Fog
Created By	System Development Team
Last Updated By	System Development Team
Date Created	2026
Date Last Updated	2026
Actor	Vehicle Sensors
Description	The edge device transmits collected sensor data to the fog node for further processing and analysis.
Preconditions	Sensor data has been collected successfully.
Postconditions	Sensor data reaches the fog node.
Priority	High
Frequency of Use	Continuous
Normal Course of Events	1. Edge device packages sensor data. 2. Data is transmitted to fog node. 3. Fog node confirms reception.
Alternative Courses	Network latency → system buffers data.
Exceptions	Network connection failure.
Includes	Process Sensor Data
Special Requirements	Reliable wireless communication protocol.
Assumptions	Network connectivity exists between edge device and fog node.
Notes and Issues	Data packet loss must be minimized.

UC-03: Process Sensor Data

Field	Description
Use Case ID	UC-03
Use Case Name	Process Sensor Data
Created By	System Development Team
Last Updated By	System Development Team
Date Created	2026
Date Last Updated	2026
Actor	System
Description	The fog node preprocesses incoming sensor data by filtering noise and normalizing values.
Preconditions	Sensor data has been received.
Postconditions	Clean and structured sensor data is generated.
Priority	High
Frequency of Use	Continuous
Normal Course of Events	1. Fog node receives sensor data. 2. Noise filtering is applied. 3. Processed data is stored for feature extraction.
Alternative Courses	Data inconsistency detected → data correction applied.
Exceptions	Data corruption occurs.
Includes	Generate Health Vector

Special Requirements	Low processing latency.
Assumptions	Fog node has sufficient processing capability.
Notes and Issues	Processing delays may affect system response time.

UC-04: Run Fault Detection

Field	Description
Use Case ID	UC-04
Use Case Name	Run Fault Detection
Actor	System
Description	The AI model analyzes vehicle health data to detect faults or abnormal conditions.
Preconditions	Health vector has been generated.
Postconditions	Fault classification results are produced.
Priority	High
Frequency of Use	Continuous
Normal Course of Events	1. Health vector is input to AI model. 2. Model processes data. 3. Fault status is determined.
Alternative Courses	Model confidence low → request further analysis.
Exceptions	AI model failure.
Includes	Generate Safety Decision
Special Requirements	High prediction accuracy.

Assumptions	AI model is trained and available.
Notes and Issues	Incorrect predictions may cause false alerts.

UC-05: Generate Safety Decision

Field	Description
Use Case ID	UC-05
Use Case Name	Generate Safety Decision
Actor	System
Description	The system determines whether safety action is required based on detected faults.
Preconditions	Fault detection results are available.
Postconditions	Safety decision is generated.
Priority	Critical
Frequency of Use	When abnormal condition detected
Normal Course of Events	1. System evaluates fault severity. 2. Safety threshold is checked. 3. Safety decision is generated.
Alternative Courses	Non-critical fault → monitoring continues.
Exceptions	Incomplete data prevents decision making.
Includes	Send Actuation Command
Special Requirements	Fast decision making.
Assumptions	Decision thresholds are predefined.

Notes and Issues	Incorrect threshold values may trigger false alarms.
------------------	--

UC-06: Send Actuation Command

Field	Description
Use Case ID	UC-06
Use Case Name	Send Actuation Command
Actor	Actuator System
Description	The system sends control commands to the vehicle actuator to mitigate detected faults.
Preconditions	Safety decision indicates critical condition.
Postconditions	Actuator performs corrective action.
Priority	Critical
Frequency of Use	Rare (only during faults)
Normal Course of Events	1. The system generates actuation commands. 2. Command sent to the actuator system. 3. Actuator executes corrective action.
Alternative Courses	Actuator busy → command queued.
Exceptions	Actuator malfunction.
Includes	—
Special Requirements	Immediate response required.
Assumptions	The actuator system is operational.
Notes and Issues	Safety mechanisms must prevent unintended commands.

UC-07: Upload Health Data

Field	Description
Use Case ID	UC-07
Use Case Name	Upload Health Data
Actor	Cloud Service
Description	Vehicle health data is uploaded to cloud servers for storage and analytics.
Preconditions	Health vector generated.
Postconditions	Data stored in a cloud database.
Priority	Medium
Frequency of Use	Periodic
Normal Course of Events	1. Fog node prepares data. 2. Data transmitted to cloud. 3. Cloud stores the data.
Alternative Courses	Network delay → data buffered.
Exceptions	Cloud service unavailable.
Includes	Store Vehicle History
Special Requirements	Secure data transmission.
Assumptions	Cloud connectivity exists.
Notes and Issues	Data privacy must be maintained.

UC-08: View Vehicle Health

Field	Description
-------	-------------

Use Case ID	UC-08
Use Case Name	View Vehicle Health
Actor	Vehicle Driver, Maintenance Operator
Description	Users can view the current health status of the vehicle through the dashboard.
Preconditions	Vehicle health data available.
Postconditions	Health information displayed.
Priority	Medium
Frequency of Use	Frequent
Normal Course of Events	1. User accesses dashboard. 2. System retrieves latest data. 3. Vehicle health displayed.
Alternative Courses	Data unavailable → system displays previous records.
Exceptions	Dashboard service failure.
Includes	Receive Alerts
Special Requirements	User-friendly interface.
Assumptions	Dashboard system operational.
Notes and Issues	Visualization accuracy is important.

UC-09: Receive Alerts

Field	Description
Use Case ID	UC-09
Use Case Name	Receive Alerts

Actor	Vehicle Driver, Maintenance Operator
Description	System notifies users when abnormal vehicle conditions occur.
Preconditions	Fault detected by system.
Postconditions	Alert displayed to users.
Priority	High
Frequency of Use	When faults occur
Normal Course of Events	1. Fault detected. 2. Alert generated. 3. Alert displayed on dashboard.
Alternative Courses	Notification delayed due to network latency.
Exceptions	Notification service failure.
Includes	—
Special Requirements	Real-time notifications.
Assumptions	Dashboard accessible to users.
Notes and Issues	Excess alerts may cause user fatigue.

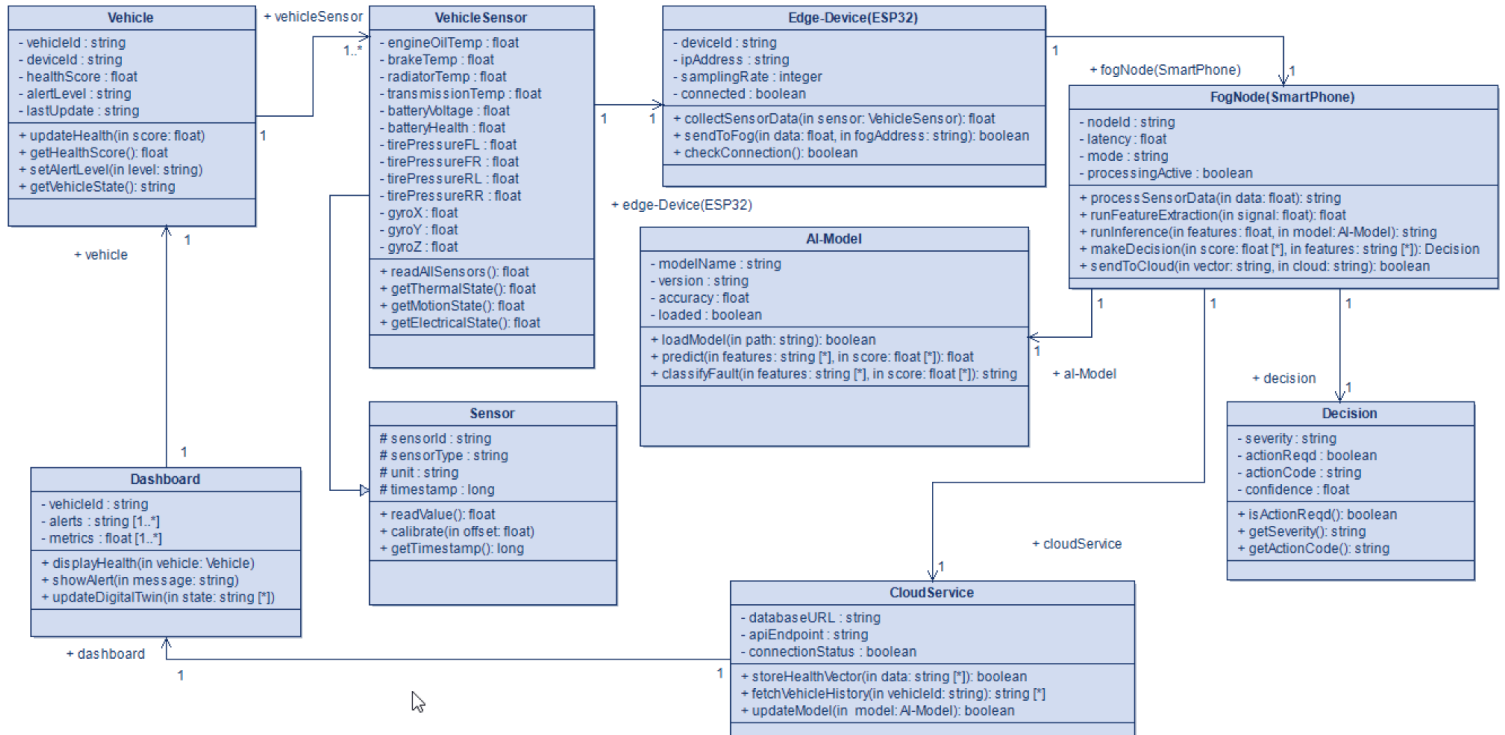
UC-10: Update Digital Twin

Field	Description
Use Case ID	UC-10
Use Case Name	Update Digital Twin
Actor	Dealership Maintenance Operator

Description	Updates the digital twin model of the vehicle with latest operational data.
Preconditions	Vehicle telemetry available.
Postconditions	Digital twin reflects latest vehicle state.
Priority	Medium
Frequency of Use	Periodic
Normal Course of Events	1. Operator accesses digital twin system. 2. System retrieves latest data. 3. Digital twin model updated.
Alternative Courses	Data unavailable → update postponed.
Exceptions	Digital twin system failure.
Includes	—
Special Requirements	Accurate simulation model.
Assumptions	Cloud data accessible.
Notes and Issues	Model synchronization required.

Class Diagram

[Class Diagram Link](#)



CRC Card

[CRC Card Link](#)

Vehicle		Sensor		VehicleSensor	
Responsibilities	Collaborators	Responsibilities	Collaborators	Responsibilities	Collaborators
- Maintain the overall health state of the vehicle - Store health score calculated by AI - Maintain alert level - Provide vehicle state snapshot for dashboard - Track latest update timestamp - Link vehicle with edge device	- VehicleSensor → provides sensor measurements - Dashboard → displays vehicle health - CloudService → stores vehicle history - Decision → updates alert level	- Define generic sensor interface - Provide common attributes - Provide timestamp tracking - Support sensor calibration	- VehicleSensor - EdgeDevice	- Aggregate all vehicle telemetry - Provide categorized sensor states - Thermal - Motion - Electrical - Perform raw sensor acquisition - Supply structured data for AI inference	- Sensor - EdgeDevice - FogNode
EdgeDevice (ESP32)		FogNode (Smartphone)		AIModel	
Responsibilities	Collaborators	Responsibilities	Collaborators	Responsibilities	Collaborators
- Collect sensor data from VehicleSensor - Manage hardware communication - Perform basic validation - Send data to Fog Node - Monitor network connectivity - Manage sampling rate	- VehicleSensor - FogNode	- Receive sensor data - Preprocess signals - Extract features - Run AI inference - Generate safety decisions - Send processed vector to cloud - Maintain processing state	- EdgeDevice - AIModel - Decision - CloudService	- Load trained ML model - Perform inference - Predict vehicle health score - Classify fault type - Provide prediction confidence	- FogNode - CloudService (model updates)
Decision		CloudService		Dashboard	
Responsibilities	Collaborators	Responsibilities	Collaborators	Responsibilities	Collaborators
- Represent AI decision output - Determine severity level - Specify required action - Provide action code	- FogNode - Vehicle - Dashboard	- Store vehicle telemetry - Maintain vehicle history - Provide analytics backend - Provide model updates - Support API endpoints	- FogNode - AIModel - Dashboard	- Display vehicle health - Display alerts - Show digital twin - Visualize metrics - Query vehicle history	- Vehicle - CloudService - Decision

Code Generation using class diagram.

```
Sensor.java UNREGISTERED
Vehicle.java x VehicleSensor.java x Sensor.java x EdgeDeviceESP32.java x ArtPhone.java x ||| + v
1 public class Sensor {
2
3     protected String sensorId;
4     protected String sensorType;
5     protected String unit;
6     protected long timestamp;
7
8     public float readValue() {
9         return 0;
10    }
11
12    public void calibrate(float offset) {
13    }
14
15    public long getTimestamp() {
16        return timestamp;
17    }
18 }
```

```
FogNodeSmartPhone.java UNREGISTERED
Vehicle.java x VehicleSensor.java x Sensor.java x EdgeDeviceESP32.java x FogNodeSmartPhone.java x AIModel.java x Service.java x + v
1 public class FogNodeSmartPhone {
2
3     private String nodeId;
4     private float latency;
5     private String mode;
6     private boolean processingActive;
7
8     public String processSensorData(float data) {
9         return "";
10    }
11
12    public String runFeatureExtraction(float data) {
13        return "";
14    }
15
16    public String runInference(String features) {
17        return "";
18    }
19
20    public String makeDecision(float score) {
21        return "";
22    }
23
24    public boolean sendToCloud(String vector, String cloudAddress) {
25        return false;
26    }
27 }
```

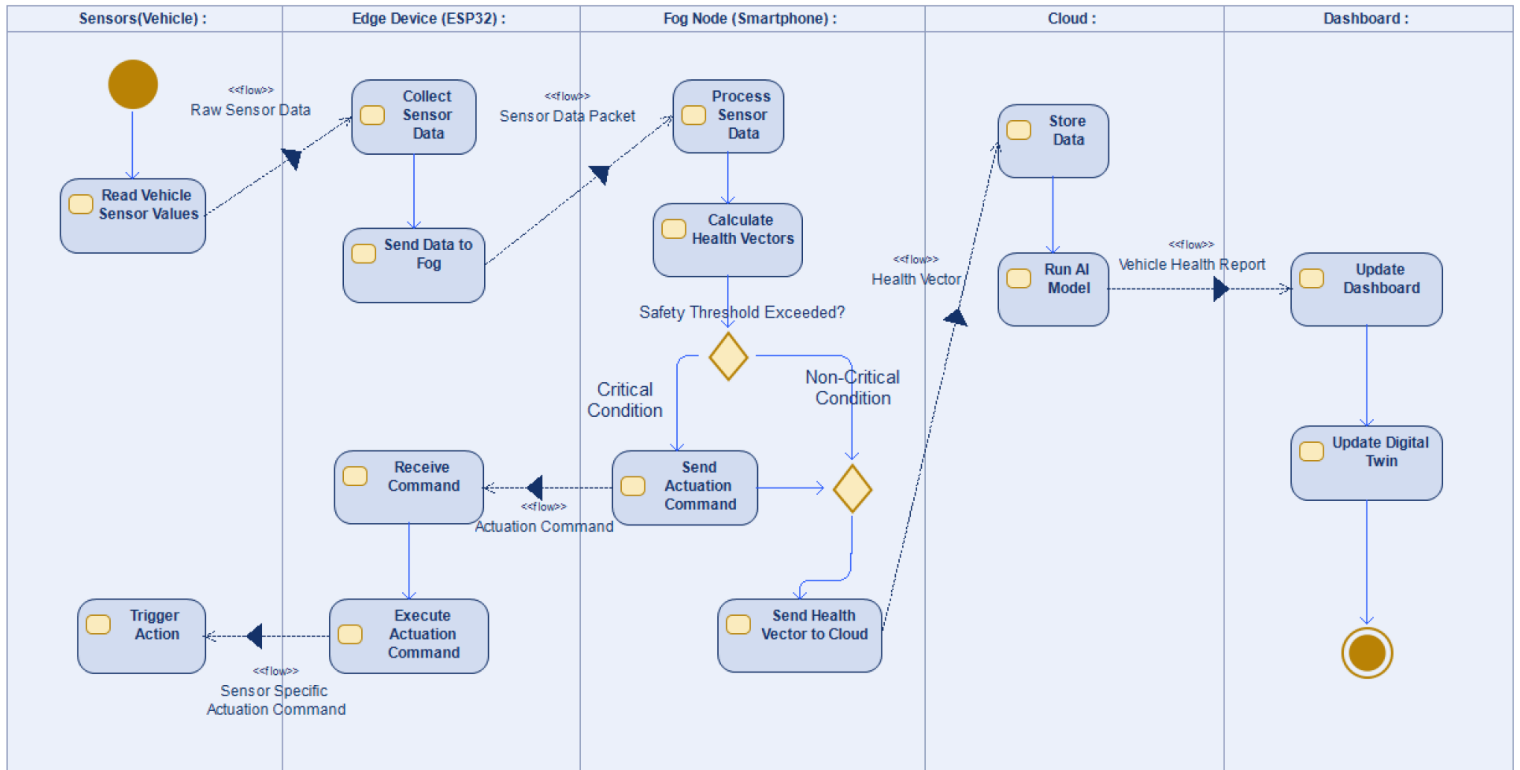
```
AIModel.java UNREGISTERED
Vehicle.java x VehicleSensor.java x Sensor.java x EdgeDeviceESP32.java x FogNodeSmartPhone.java x AIModel.java x Service.java x + v
1 public class AIModel {
2
3     private String modelName;
4     private String version;
5     private float accuracy;
6     private boolean loaded;
7
8     public boolean loadModel(String path) {
9         return true;
10    }
11
12    public float predict(String[] features) {
13        return 0;
14    }
15
16    public String classifyFault(String[] features, float score) {
17        return "";
18    }
19 }
```

```
CloudService.java UNREGISTERED
Sensor.java EdgeDeviceESP32.java FogNodeSmartPhone.java AIModel.java CloudService.java Dashboard.java Decision.java +
1 public class CloudService {
2
3     private String databaseURL;
4     private String apiEndpoint;
5     private boolean connectionStatus;
6
7     public boolean storeHealthVector(String[] data) {
8         return true;
9     }
10
11    public String[] fetchVehicleHistory(String vehicleId) {
12        return new String[0];
13    }
14
15    public boolean updateModel(String modelPath) {
16        return true;
17    }
18 }
```

```
Dashboard.java UNREGISTERED
Sensor.java EdgeDeviceESP32.java FogNodeSmartPhone.java AIModel.java CloudService.java Dashboard.java Decision.java +
1 public class Dashboard {
2
3     private String vehicleId;
4     private String[] alerts;
5     private String[] metrics;
6
7     public void displayHealth(Vehicle vehicle) {
8     }
9
10    public void showAlert(String message) {
11    }
12
13    public void updateDigitalTwin(String[] state) {
14    }
15 }
```

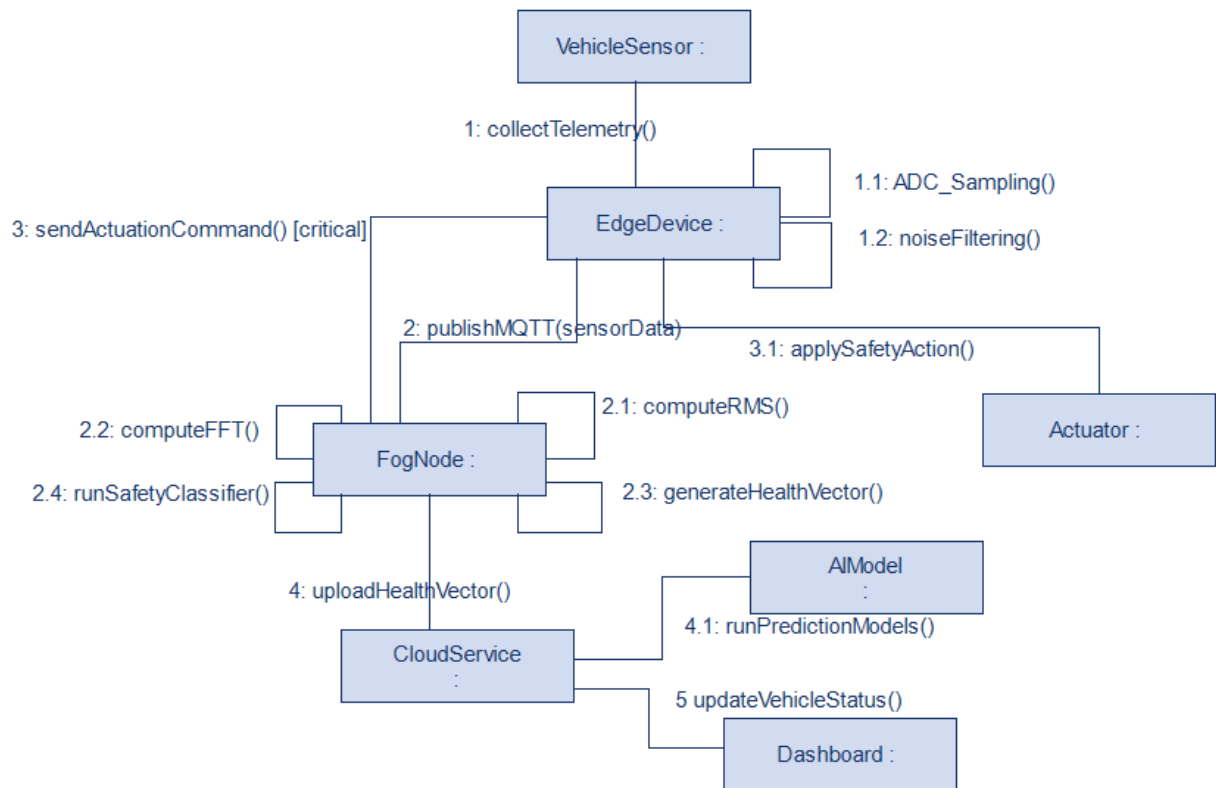
```
Decision.java UNREGISTERED
Sensor.java EdgeDeviceESP32.java FogNodeSmartPhone.java AIModel.java CloudService.java Dashboard.java Decision.java +
1 public class Decision {
2
3     private String severity;
4     private String actionRequired;
5     private long timestamp;
6
7     public String evaluateSeverity(float healthScore) {
8         return "";
9     }
10
11    public String recommendedAction() {
12        return "";
13    }
14 }
```


Activity Diagram



[Activity Diagram Link](#)

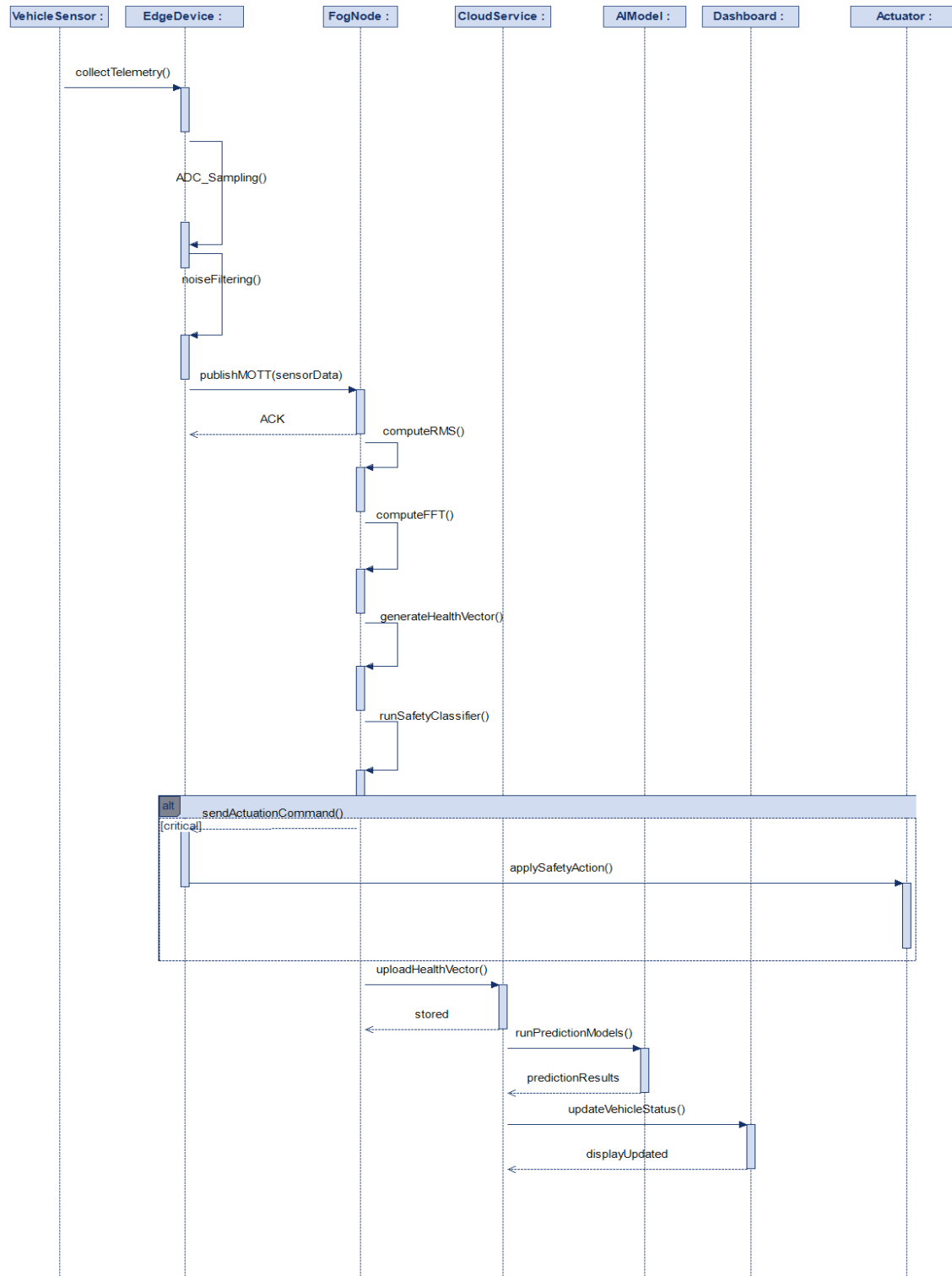
Communication/Collaboration Diagram



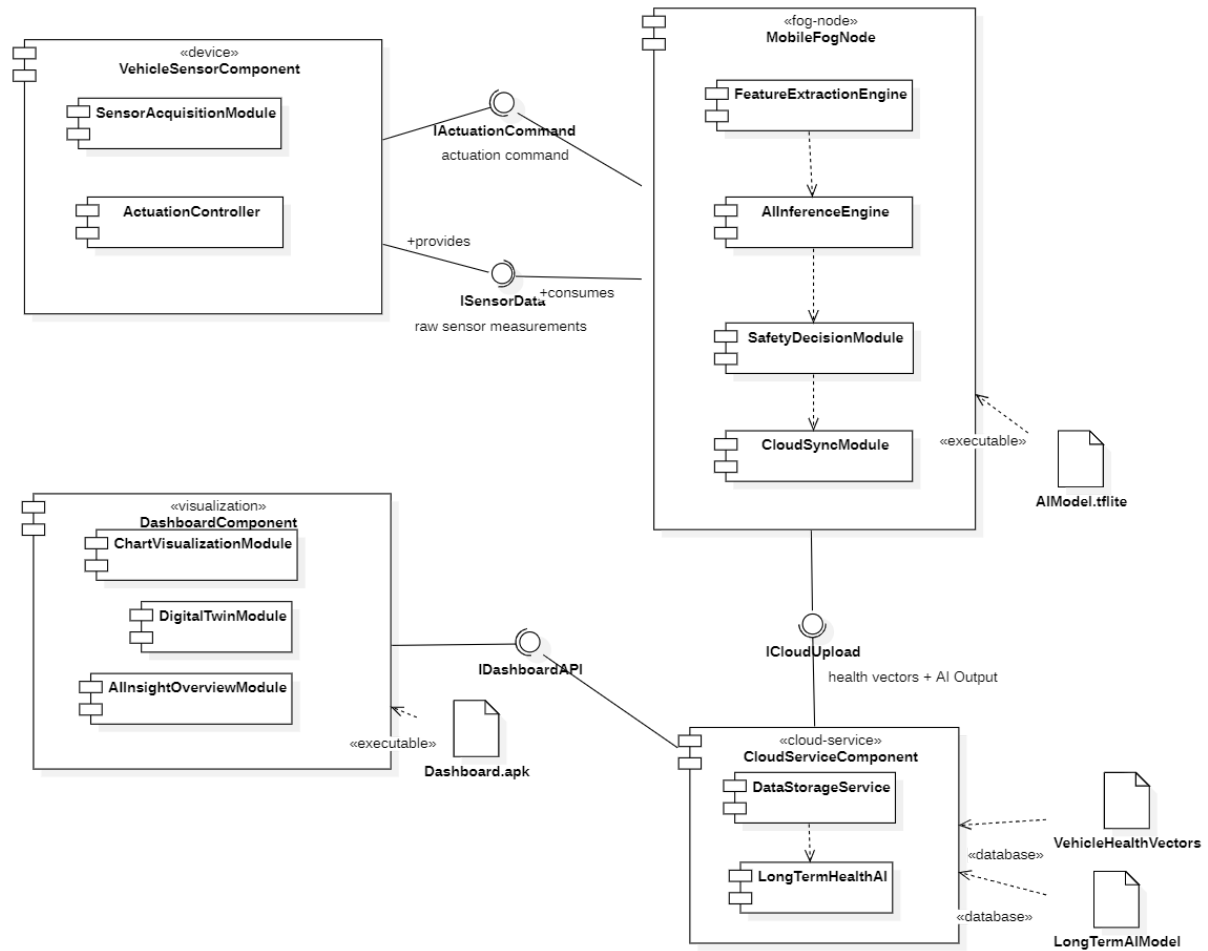
[Communication/Collaboration Diagram Link](#)

Sequence Diagram

[Sequence Diagram Link](#)

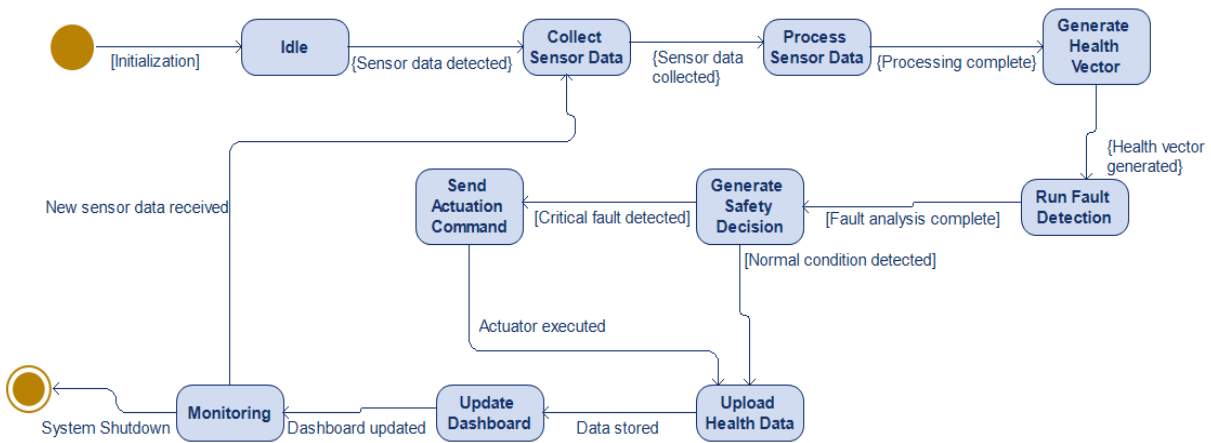


Component Diagram



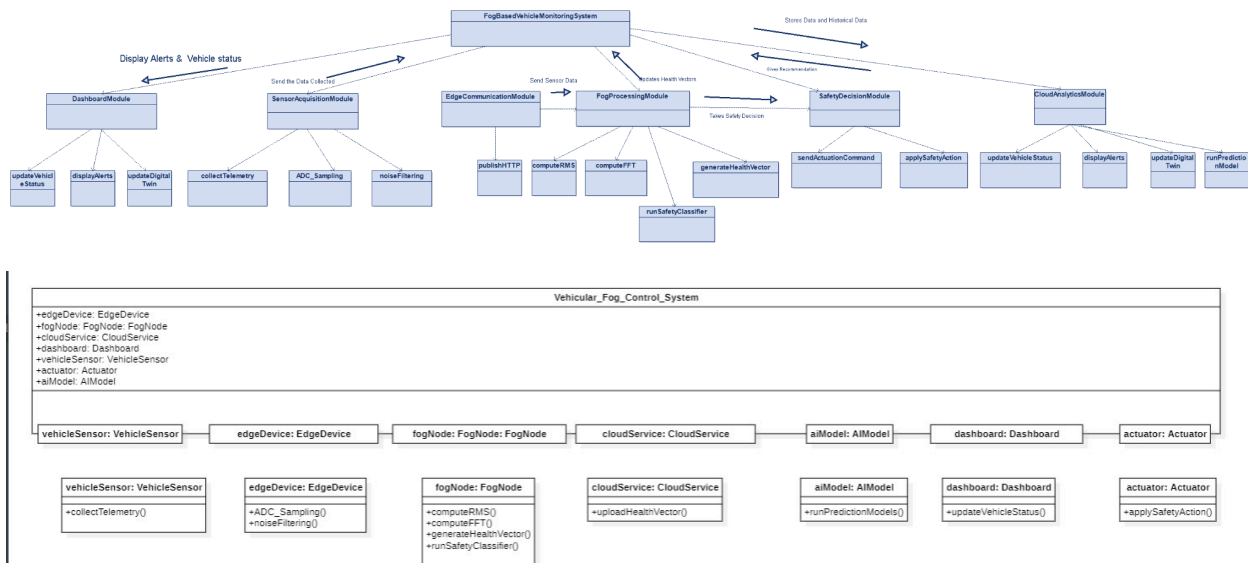
[Link to the Component Diagram.](#)

Statechart Diagram:



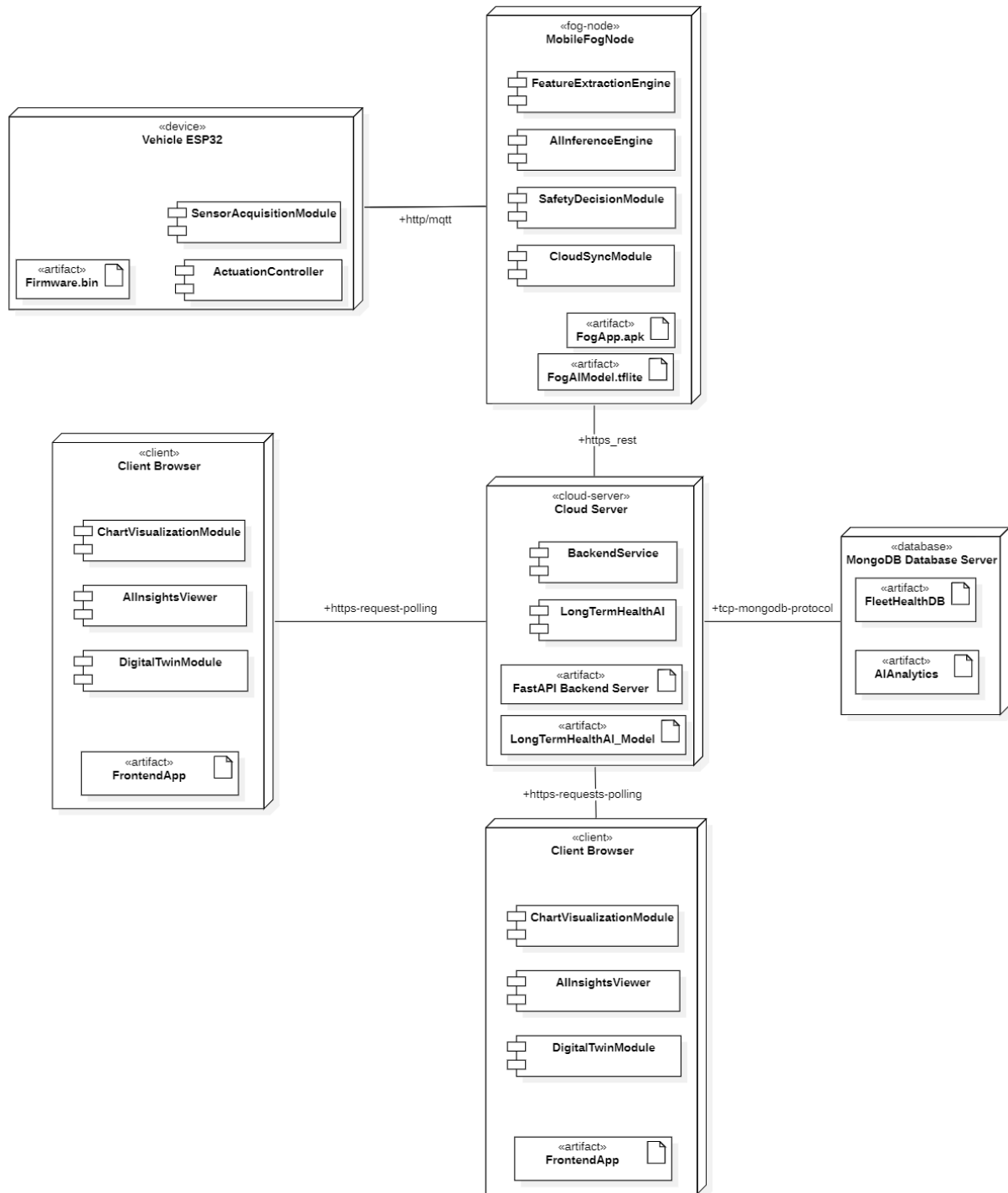
[Statechart Diagram Link](#)

Structure Chart:



[Structure Chart Link](#)

Deployment Diagram:



[Link to the deployment diagram.](#)